Terna Engineering College

Computer Engineering Department
Program: Sem VI

Course: Cloud Computing Lab(CSL605)

PART A
(PART A: TO BE REFFERED BY STUDENTS)

Experiment No.10

A.1 Aim:

To study and implement container orchestration using Kubernetes .

A.2 Prerequisite:

Knowledge of container technology, Docker, basics of node.js

A.3 Objective:

To understand the steps to deploy Kubernetes Cluster on local systems, deploy applications on Kubernetes, creating a Service in Kubernetes, develop Kubernetes configuration files in YAML and creating a deployment in Kubernetes using YAML

A.4 Outcome: (LO 4)

After successful completion of this experiment student will be able to, Understand the concept of Kubernetes cluster.

A.5 Theory:

Container orchestration tools provide a framework for managing containers and microservices architecture at scale. There are many container orchestration tools that can be used for container lifecycle management. Some popular options are Kubernetes, Docker Swarm, and Apache Mesos.

Kubernetes orchestration allows you to build application services that span multiple containers, schedule containers across a cluster, scale those containers, and manage their health over time.

Kubernetes eliminates many of the manual processes involved in deploying and scaling containerized applications.

Main components of Kubernetes:

- Cluster: A control plane and one or more
- compute machines, or nodes.

Control plane: The collection of processes that control Kubernetes nodes. This is where all task assignments originate.

- Kubelet: This service runs on nodes and reads the container manifests and ensures the defined containers are started and running.
- Pod: A group of one or more containers deployed to a single node. All containers in a pod share an IP address, IPC, hostname, and other resources.

The following instructions show you how to set up a simple, single node Kubernetes cluster using Docker.

1. Create a Kubernetes cluster.
2. Deploy an app.
3. Explore your app.
4. Expose your app publicly.
5. Scale up your app.
6. Update your app.

Steps:

1. Open the terminal on Ubuntu.

2. Install the necessary dependencies by using the following command:

```
$ sudo apt-get update

$ sudo apt-get install -y apt-transport-https
```

3. Install Docker Dependency by using the following command:

```
$ sudo apt install docker.io
```

Start and enable Docker with the following commands:

```
$ sudo systemctl start docker

$ sudo systemctl enable docker
```

4. Install the necessary components for Kubernetes.

First, install the curl command:

```
$ sudo apt-get install curl
```

Then download and add the key for the Kubernetes install:

```
$ sudo curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add
```

Change permission by using the following command:

```
$ sudo chmod 777 /etc/apt/sources.list.d/
```

Then, add a repository by creating the file /etc/apt/sources.list.d/kubernetes.list and enter the following content:

deb http://apt.kubernetes.io/ kubernetes-xenial main Save

and close that file.

Install Kubernetes with the following commands:

```
$ apt-get update

$ apt-get install -y kubelet kubeadm kubectl kubernetes-cni
```

5. Before initializing the master node, we need to swap off by using the following command:

```
$ sudo swapoff −a
```

6. Initialize the master node using the following command:

```
$ sudo kubeadm init
```

You get three commands: copy and paste them and press and “enter.”

```
$ mkdir -p $HOME/.kube

$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config

$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

7. Deploy pods using the following command:

```
$ $ sudo kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/ master/Documentation/kube-flannel.yml
```

```
$ sudo kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/ master/Documentation/k8s-manifests/kube-flannel-rbac.yml
```

8. To see all pods deployed, use the following command:

```
$ sudo kubectl get pods –all-namespaces
```

9. To deploy an NGINX service (and expose the service on port 80), run the following commands:

```
$ sudo kubectl run --image=nginx nginx-app --port=80 --env="DOMAIN=cluster"

$ sudo kubectl expose deployment nginx-app --port=80 --name=nginx-http
```

10. To see the services listed, use the following command:

```
$ sudo docker ps –a
```

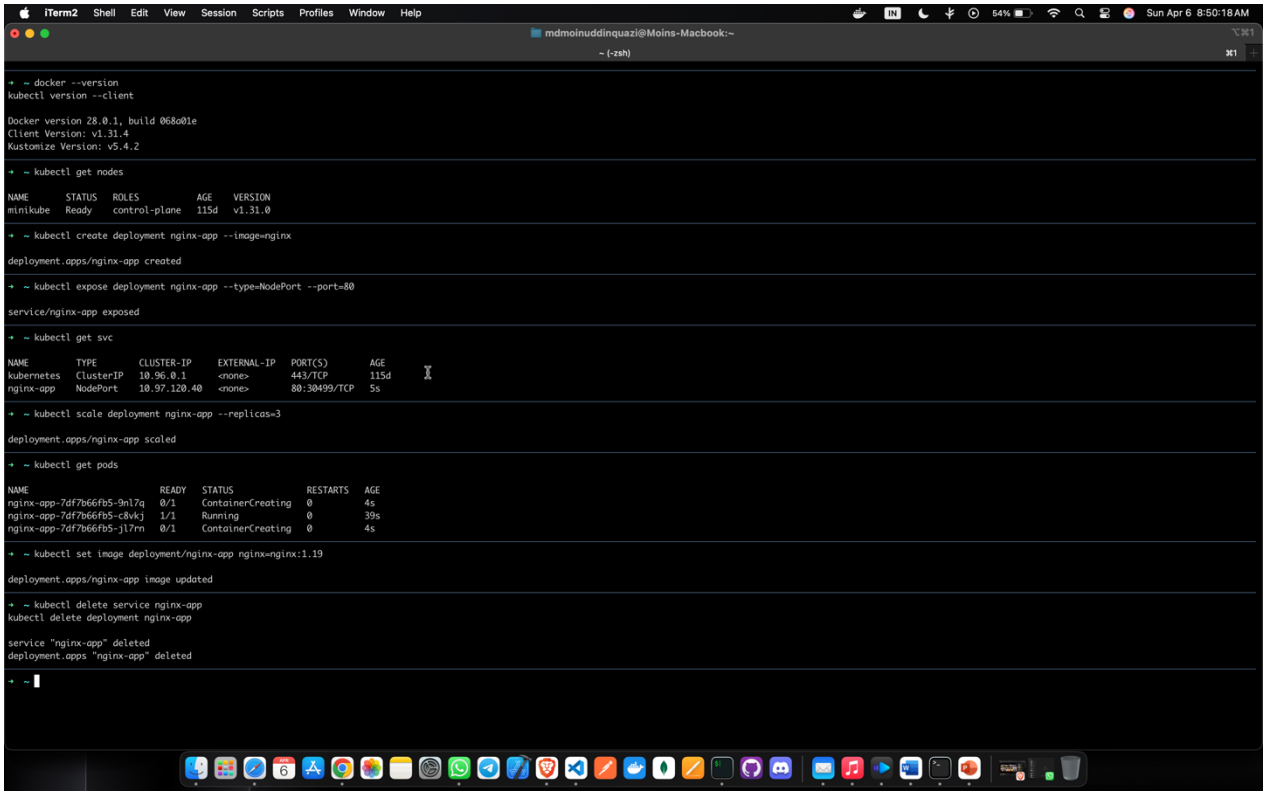
PART B

(PART B: TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded on the ERP or emailed to the concerned lab in charge faculties at the end of the practical in case the there is no ERP access available)

Roll No : C54	Name: Quazi Md Moinuddin
Class : TE-C	Batch : C3
Date of Experiment:3-4-2025	Date of Submission : 5-4-2025
Grade :	

Screenshots:



B.1 Question of Curiosity:

(To be answered by student based on the practical performed and learning/observations) Q.1 What is Kubernetes?

Ans Kubernetes, often abbreviated as K8s, is an open-source platform designed to automate the deployment, scaling, and management of containerized applications. Originally developed by Google, Kubernetes has gained immense popularity and is now maintained by the Cloud Native Computing Foundation (CNCF), a Linux Foundation project.At its core, Kubernetes provides a container orchestration framework that abstracts away the underlying infrastructure, enabling developers to focus on building and deploying applications using containers. Containers encapsulate application code along with its dependencies, providing consistency and portability across different environments.

Key components of Kubernetes include:

1. Pods: The smallest deployable unit in Kubernetes, representing one or more containers that share networking and storage resources.
2. Services: Abstracts away individual Pods and provides a consistent way to access them. Services enable load balancing and service discovery within the Kubernetes cluster.
3. Deployments: Provides declarative updates for Pods and ReplicaSets, facilitating the deployment and scaling of applications.
4. ReplicaSets: Ensures that a specified number of identical Pods are running at all times, allowing for easy scaling and high availability.
5. Nodes: Physical or virtual machines that run the Kubernetes components and host Pods.
6. Master Node: Manages the Kubernetes cluster and orchestrates communication between nodes.

7. Worker Node: Executes Pods and provides necessary runtime environments, such as Docker or containerd.
8. Controllers: Monitor the state of various Kubernetes objects and ensure that the desired state matches the actual state.
9. Volumes: Provides persistent storage to Pods, allowing data to persist across container restarts.
10. ConfigMaps and Secrets: Store configuration data and sensitive information securely, allowing applications to access them as environment variables or mounted files.

Q.2 What are the features of Kubernetes?

Ans Kubernetes offers a wide range of features that enable efficient management and orchestration of containerized applications. Some key features include:

1. Container Orchestration: Kubernetes automates the deployment, scaling, and management of containerized applications, ensuring consistent performance and reliability.

2. Service Discovery and Load Balancing: Kubernetes automatically assigns network addresses to containers and load balances traffic across them, facilitating seamless communication between services.
3. Automatic Scaling: Kubernetes can automatically scale applications based on resource usage metrics, such as CPU and memory utilization, to handle varying levels of traffic and demand.
4. Self-Healing: Kubernetes continuously monitors the health of applications and containers. If a container fails or becomes unresponsive, Kubernetes automatically restarts it or replaces it with a new instance, ensuring high availability.

5. Rolling Updates and Rollbacks: Kubernetes supports rolling updates, allowing new versions of applications to be deployed gradually without downtime. In case of issues, it also facilitates rollback to a previous stable version.
6. Storage Orchestration: Kubernetes provides persistent storage solutions and manages storage volumes for stateful applications, ensuring data persistence and availability.
7. Resource Isolation and Management: Kubernetes allocates and manages compute resources, such as CPU and memory, for containers, ensuring efficient resource utilization and isolation.
8. Multi-Tenancy Support: Kubernetes supports multiple users or teams within the same cluster, providing isolation and resource allocation mechanisms to ensure fair resource sharing.
9. Configuration Management: Kubernetes enables the configuration of applications through configuration files or declarative manifests, allowing for easy management and version control of application configurations.
10. Secrets Management: Kubernetes allows secure storage and management of sensitive information, such as passwords and API tokens, ensuring that they are accessible only to authorized applications.

11. Role-Based Access Control (RBAC): Kubernetes provides fine-grained access control mechanisms, allowing administrators to define roles and permissions for users and service accounts.
12. Monitoring and Logging: Kubernetes integrates with monitoring and logging solutions, allowing administrators to monitor the health and performance of applications and infrastructure components.

Q.3 What are the different services within Kubernetes?

Ans Within Kubernetes, there are several key services that work together to enable container orchestration and management. These services include:

1. API Server: The Kubernetes API server acts as the primary interface for interacting with the Kubernetes cluster. It handles API requests, performs authentication and authorization, and validates and processes resource configuration.
2. Scheduler: The Kubernetes scheduler is responsible for assigning Pods to Nodes based on resource requirements, quality of service requirements, and other constraints specified in the Pod's configuration.
3. Controller Manager: The controller manager is a collection of controllers that manage various aspects of the cluster's state. These controllers include the ReplicaSet controller, which ensures that the correct number of Pods are running, the Deployment controller, which manages rolling updates and rollbacks of Deployments, and other controllers for managing endpoints, services, and namespaces.
4. etcd: etcd is a distributed key-value store that serves as the Kubernetes cluster's backing store for all cluster data. It stores configuration data, state information, and metadata about cluster objects.
5. Kubelet: The Kubelet is an agent that runs on each Node in the cluster and is responsible for managing the Pods running on that Node. It communicates with the API server to receive Pod specifications, pull container images, and start, stop, and monitor containers.
6. Kube-proxy: Kube-proxy is a network proxy that runs on each Node in the cluster and maintains network rules to enable communication between Pods and external clients. It provides load balancing for Services and handles routing of network traffic.
7. Container Runtime: Kubernetes supports multiple container runtimes, including Docker, containerd, and CRI-O. The container runtime is responsible for managing container lifecycle, including starting, stopping, and managing container processes.

8. Cloud Controller Manager: The cloud controller manager runs cloud-specific control loops that interact with the underlying cloud provider's APIs. It manages cloud resources such as load balancers, volumes, and instances, and integrates with Kubernetes to provision and manage these resources.

Q.4.What is ClusterIP , NodePort, LoadBalancer in kubernets?

Ans In Kubernetes, ClusterIP, NodePort, and LoadBalancer are three different types of services used to expose applications running inside the cluster to external clients or other services. Each type has its own use case and provides different levels of access and functionality:

1. ClusterIP:
   - ClusterIP is the default type of service in Kubernetes.
   - It exposes the Service on a cluster-internal IP, which is accessible only from within the cluster.
   - This type of service is typically used for communication between different components or services within the cluster.
   - External clients cannot access services with ClusterIP directly; they need to go through other types of services or Ingress resources.

2. NodePort:
   - NodePort exposes the Service on a port on each node in the cluster.
   - It creates a listening port on each node's IP address, and any traffic sent to this port is forwarded to the Service.
   - NodePort services are accessible from outside the cluster by accessing any node's IP address and the specified NodePort.
   - This type of service is useful for exposing applications that require direct access from outside the cluster, but it's typically not recommended for production use due to security and scalability concerns.

3. LoadBalancer:

- LoadBalancer automatically provisions an external load balancer from the cloud provider's load balancer service.
- It assigns a unique external IP address to the Service, and the external load balancer routes traffic to the Service's ClusterIP.
- LoadBalancer services are suitable for exposing applications to external clients with high availability and scalability requirements.
- This type of service is commonly used in production environments to expose web applications, APIs, or other services to the internet.

B.3 Conclusion:

In conclusion, studying and implementing container orchestration using Kubernetes offers a deep understanding of modern application deployment and management practices. It provides insights into scalable, resilient, and automated infrastructure management, enabling efficient utilization of resources and seamless deployment of containerized applications. Overall, Kubernetes empowers organizations to build, deploy, and manage cloudnative applications with ease, ensuring agility, reliability, and scalability in dynamic environments.